

Recreating Our Meta App From Scratch

A step-by-step guide to building a Meta (Facebook + Instagram) developer app functionally identical to the one PostAutomation runs in production — using your own developer account.

APP TYPE Business	APP MODE Live
GRAPH API VERSION v18.0	PRODUCTS FB Login for Business · Instagram · Pages API
PERMISSIONS NEEDING REVIEW 9 (6 publishing + 3 insights)	REALISTIC END-TO-END TIME 6–10 weeks

Read §13 (Timeline) before you begin. The dashboard work takes about two hours; the Meta-side waits — Business Verification, Data Access Renewal, then two separate App Review submissions — are largely sequential, because each one un-greys the button for the next. Sequencing them wrong costs weeks.

Recreating Our Meta App From Scratch

Everything below is derived from the live app's actual configuration and from the code that consumes it — not from generic Meta documentation. Where a value is arbitrary (your app name, your domain) it is marked `<your-value>` .

Time to complete: ~2 hours of dashboard work, then **3–8 weeks of waiting** on Meta (Business Verification → Data Access Renewal → App Review x2). The waiting is unavoidable and mostly sequential. Read [§13 Timeline](#) before you start so you sequence it correctly.

Table of contents

#	PHASE	BLOCKING?
0	What you are recreating (target spec)	—
1	Prerequisites	⚠️ Yes
2	Create the app	—
3	Add products	—
4	Basic settings + compliance URLs	⚠️ Gates App Review
5	Turn OFF "Native or desktop app"	⚠️ Breaks OAuth if wrong
6	Redirect URLs	⚠️ Breaks OAuth if wrong
7	Business Verification	⚠️ Gates App Review
8	Wire credentials into your app	—
9	Satisfy the test-call gate	⚠️ Gates App Review
10	Data Access Renewal	⚠️ Gates App Review
11	App Review #1 — publishing (6 perms)	—
12	App Review #2 — insights (3 perms)	—
13	Timeline	—
14	Traps — do NOT do this	⚠️ Read this
15	Reference: permission → API call map	—
16	Verification checklist	—

0 What you are recreating

This is the target. Every row is the live production value — check your finished app against this table.

SETTING	VALUE
App name	<code><your-app-name></code> (ours: <code>Post Automation 2</code>)
App type	Business
App mode	Live (not Development)
Business Verification	✅ Verified
Tech Provider Verification	✅ Verified
Products	Facebook Login for Business · Instagram · Pages API
Graph API version	<code>v18.0</code>

SETTING	VALUE
"Native or desktop app?"	OFF
Login "Strict Mode" for redirect URIs	ON
"Enforce HTTPS"	ON
Client OAuth Login	ON
Web OAuth Login	ON
Facebook permissions requested	public_profile , pages_show_list , pages_manage_posts , pages_read_engagement , pages_read_user_content , read_insights
Instagram permissions requested	public_profile , pages_show_list , pages_read_engagement , instagram_basic , instagram_content_publish , business_management , instagram_manage_insights
Distinct permissions needing App Review	9 (6 publishing + 3 insights)
Deliberately NOT requested	instagram_manage_comments , email , any post_impressions* metric

Authoritative source in code: `getDefaultScopes()` in `channel.router.ts:480-527` . If you change the scope arrays there, this table must change with them.

1 Prerequisites (do these first)

Do not open the Meta dashboard until all five are true. Missing any of them will stall you mid-way with a half-configured app.

1.1 A Facebook account with 2FA enabled

Meta requires two-factor auth on any account that administers an app requesting advanced permissions. Enable it at facebook.com/settings?tab=security.

1.2 A Meta Business Portfolio (formerly Business Manager)

Create one at business.facebook.com/overview. A Business-type app must be owned by a business portfolio — a personal account alone cannot complete Business Verification. You will need real, verifiable business details: legal entity name, registered address, business phone, and a business email on your own domain.

1.3 A Facebook Page you administer

Instagram publishing is routed through a Page (see the insight at the top of this guide). Without a Page there is nothing to post to and your own testing will fail with `fb_no_pages` .

Create one at facebook.com/pages/create.

1.4 An Instagram Professional/Business account linked to that Page

In the Instagram mobile app: **Settings** → **Account type and tools** → **Switch to professional account**, then **Settings** → **Sharing to other apps** → **Facebook** and link it to the Page from §1.3.

Verify the link resolved correctly — the Page must show the IG account under **Page Settings** → **Linked accounts**. Our code resolves IG through `me/accounts` → `instagram_business_account` , so an unlinked IG account is invisible to the API and connect fails with `ig_no_business_account` .

1.5 Three live, publicly reachable compliance pages on your own HTTPS domain

PAGE	OURS	MUST BE
Privacy Policy	https://postautomation.co.in/privacy	HTTP 200, no auth wall
Terms of Service	https://postautomation.co.in/terms	HTTP 200, no auth wall
Data Deletion Instructions	https://postautomation.co.in/data-deletion	HTTP 200, no auth wall

Meta's reviewers fetch these. A 404, a redirect loop, or a login gate is a rejection. Ours are plain Next.js pages — see `apps/web/app/data-deletion/page.tsx` for the shape of the data-deletion page.

Why this is a prerequisite, not a step: you cannot submit for App Review until these URLs are filled in and live, and Business Verification also checks your domain. Building them last means waiting twice.

2 Create the app

1. Go to developers.facebook.com/apps and click **Create app**.
2. **App details** — enter your app name and a contact email.
3. **Use case** — choose **Other**.

Meta's use-case wizard will otherwise pre-select a fixed product/permission bundle you'd have to undo. "Other" gives you the free-form Business app.

4. **App type** — choose **Business**. ⚠ This is not changeable later without recreating the app.
5. **Business portfolio** — select the portfolio from §1.2.
6. Click **Create app** and re-enter your password.

You now have an **App ID** (a ~15-digit number) and an **App secret**. Find both under **App settings** → **Basic**. Treat the secret like a database password.

3 Add products

App Dashboard → left sidebar → **Add Product**. Add all three:

PRODUCT	WHY
Facebook Login for Business	The OAuth flow. Note: <i>for Business</i> , not plain "Facebook Login" — the business variant is what produces the Page-selection wizard the app depends on.
Instagram	Instagram Graph API publishing + insights via the linked Page.
Pages API	Reading and posting to Pages.

After adding, your sidebar should show `Facebook Login for Bus...` and `Instagram` as expandable sections.

4 Basic settings + compliance URLs

App settings → **Basic**. Fill in every field below — App Review is blocked while any are empty.

FIELD	VALUE
App name	<code><your-app-name></code>
App contact email	A monitored inbox on your domain
Privacy Policy URL	<code>https://<your-domain>/privacy</code>
Terms of Service URL	<code>https://<your-domain>/terms</code>
User data deletion	Select Data deletion instructions URL → <code>https://<your-domain>/data-deletion</code>
App icon	1024×1024 PNG, no transparency
Category	Business and pages
Business verification	Link the portfolio from §1.2

Click **Save changes**.

5 CRITICAL: turn OFF "Native or desktop app"

App settings → **Advanced** → scroll to "Native or desktop app?" → set to **NO**.

This single toggle is the highest-consequence setting in the whole guide.

If it is **ON**, Meta assumes your app cannot safely hold a client secret and therefore **refuses the server-side token exchange**. Your OAuth flow will get all the way through user consent and then die at the code→token step with:

```
OAuthException code: 1
"the app is configured as a desktop app"
```

Our `exchangeCodeForTokens` posts `client_secret` to `graph.facebook.com/v18.0/oauth/access_token` from the server, which is exactly the pattern this toggle forbids. We hit this in production on 2026-06-02; the symptom looks like a credentials bug and sends you chasing the wrong thing for hours.

Set it to **NO** and save.

6 Facebook Login for Business → redirect URIs

Facebook Login for Business → Settings.

6.1 Toggles

SETTING	VALUE
Client OAuth login	ON
Web OAuth login	ON
Enforce HTTPS	ON
Use Strict Mode for redirect URIs	ON
Login from devices	OFF
Embedded browser OAuth login	OFF

6.2 Valid OAuth Redirect URIs

Add **exactly** these four (substituting your domain), one per line:

```
https://<your-domain>/api/oauth/callback/facebook
https://<your-domain>/api/oauth/callback/instagram
http://localhost:3000/api/oauth/callback/facebook
http://localhost:3000/api/oauth/callback/instagram
```

Three rules, all of which will silently break OAuth if violated:

- **All lowercase.** The app generates the redirect URI from `platform.toLowerCase()` in `channel.router.ts:190`. With Strict Mode ON, Meta does a byte-exact match — `.../Facebook` will not match `.../facebook`.
- **Facebook and Instagram get separate URIs**, even though they share the app. Different platform slug → different callback path.
- **No trailing slashes**, no query strings.

The `localhost` entries are for local development only. Meta permits `http://localhost` as a documented exception to Enforce HTTPS. If you don't develop locally, omit them.

Click **Save changes**.

7 Business Verification + Tech Provider Verification

Both are required before advanced permissions can be granted to the public.

7.1 Business Verification

1. Go to business.facebook.com/settings/security-center.
2. Start **Business verification**.
3. Upload documents proving your legal entity: certificate of incorporation, a utility bill or bank statement showing the registered address, and a business phone number you can answer.
4. Meta verifies your domain, phone, and email. Expect **1–3 weeks**.

Docs: [Business Verification](#)

7.2 Tech Provider Verification

Required because your app handles **other businesses'** Pages and Instagram accounts, not just your own. Meta prompts for it under **App Review** → **Requests** or **Required actions** once Business Verification is done. It asks how you use the data and typically resolves faster than Business Verification.

Start \$7.1 on day one. It is the longest single wait and everything downstream depends on it. Do the rest of this guide while it processes.

8 Wire the credentials into your app

From **App settings** → **Basic**, copy the **App ID** and **App secret** into four environment variables — the *same* pair twice:

```
# .env / .env.production
FACEBOOK_CLIENT_ID=<your-app-id>
FACEBOOK_CLIENT_SECRET=<your-app-secret>
INSTAGRAM_CLIENT_ID=<your-app-id> # same App ID
INSTAGRAM_CLIENT_SECRET=<your-app-secret> # same secret

APP_URL="https://<your-domain>" # used to build the redirect URI
```

The duplication is intentional: the OAuth layer resolves credentials by `<PLATFORM>_CLIENT_ID` convention, and Facebook and Instagram are two platforms backed by one Meta app. Template: `.env.example`.

`APP_URL` must exactly match the scheme + host you registered in §6.2 — it is the string the redirect URI is built from.

Restart your app. The Connect buttons for Facebook and Instagram should stop showing "Setup required".

9 Satisfy the test-call gate

This step surprises everyone. Meta greys out the "Request advanced access" button for a permission until your app has made **at least one successful Graph API call that actually exercises that permission**. You cannot request what you haven't demonstrably used.

So: with your app still in **Development** mode, connect your own accounts (you're an app admin, so you get every requested scope at Standard Access with no review) and run the app's real flows.

PERMISSION	GRAPH CALL THAT SATISFIES THE GATE	HOW TO TRIGGER IT IN THE APP
pages_show_list	GET /me/accounts	Connect Facebook
business_management	GET /me/accounts	Connect Facebook or Instagram
instagram_basic	GET /{page-id}?fields=instagram_business_account	Connect Instagram
pages_manage_posts	POST /{page-id}/feed · /photos · /videos	Publish a post to your Page
instagram_content_publish	POST /{ig-user-id}/media → /media_publish	Publish a post to your IG
pages_read_engagement	GET /{post-id}?fields=reactions.summary,comments.summary	Click Sync Now in Insights
pages_read_user_content	same fields call as above	Click Sync Now in Insights
read_insights	GET /{post-id}/insights?metric=post_clicks,post_video_views	Click Sync Now in Insights
instagram_manage_insights	GET /{ig-media-id}/insights?metric=reach,...	Click Sync Now in Insights

Practical sequence that covers all nine:

1. Connect Facebook → covers `pages_show_list`, `business_management`.
2. Connect Instagram → covers `instagram_basic`.
3. Publish **one non-video post to the Page** → covers `pages_manage_posts`.
4. Publish **one post to Instagram** → covers `instagram_content_publish`.

⚠ Instagram has **no draft mode**. This publishes a real, public post to your IG account. Use a throwaway/test IG account if that matters to you.

5. Wait a few minutes, then trigger an analytics sync → covers the four read permissions.

Two notes:

- **Propagation takes up to 24h.** The buttons may stay greyed for a while after a successful call. Check back the next day before concluding something is wrong.

- **The analytics reads happen in the worker process, not the web app.** If your architecture is similar, make sure the worker is actually running when you trigger the sync, or those three calls never fire.

You can also fire calls by hand in the [Graph API Explorer](#) — select your app, generate a user token with the scope, and run the call. This is the fastest way to unstick one stubborn permission.

Verify progress under **App Review** → **Permissions and Features**: each permission should show a live "Request advanced access" button instead of a greyed one.

10 Data Access Renewal

Meta requires a periodic **Data Access Renewal** — you re-affirm how you handle data for permissions you already hold (`public_profile` , `email`). **Until it is cleared, the "Submit for review" button for a new App Review request is greyed out** with:

Complete data access renewal requirements to submit for App Review

It is its **own separate submission** with its own ~10-day processing time. Clearing it only **unlocks** the App Review button — you must then go submit App Review separately. Budget for two sequential waits, not one.

Find it under **App Review** → **Requests**, or **Required actions**.

The questions are about data handling, not about your code. Ours, for reference:

QUESTION	OUR ANSWER
Do you use processors/service providers?	No
Who is the data controller?	<Your Legal Entity> , <Country>
National security / government access?	No
Additional data-handling policies?	None
Supporting document	Optional — the App Review screencast serves

It is judged mostly on those answers plus your live privacy policy URL being reachable.

11 App Review submission #1 — publishing (6 permissions)

App Review → **Requests** → **Request advanced access**, and request these six:

```
pages_show_list
pages_manage_posts
pages_read_engagement
instagram_basic
instagram_content_publish
business_management
```

`public_profile` (and `email` , if you use it) are auto-granted and need no review.

11.1 Per-permission usage descriptions

For each permission, Meta asks "How will your app use this permission?" Write a description that is **literally true of your code**. This is where submissions get rejected.

Ours got rejected in June 2026 for requesting `instagram_manage_comments` : Meta correctly determined we didn't need it, because our analytics code only reads the `comments_count` **integer** (which rides on `instagram_basic`) and never reads, creates, hides, or deletes a comment thread. Verdict: **"Disallowed Use Case (Developer Policy 1.6)."**

The lesson generalizes: **request the minimum permission that makes your feature work, and describe only what your code actually does**. Reviewers check.

Suggested framing:

PERMISSION	DESCRIPTION
<code>pages_show_list</code>	"List the Facebook Pages the user administers so they can choose which Page to connect for publishing."
<code>pages_manage_posts</code>	"Publish the text, image, and video posts the user composes in our scheduler to their selected Page."
<code>pages_read_engagement</code>	"Read reaction and comment counts on posts our app published, to show the user performance metrics. Counts only — no comment moderation. "

PERMISSION	DESCRIPTION
<code>instagram_basic</code>	"Resolve the Instagram Business account linked to the user's Page and read its username/profile picture to display the connected channel."
<code>instagram_content_publish</code>	"Publish the image, video, and carousel posts the user composes to their Instagram Business account."
<code>business_management</code>	"Resolve which Pages and Instagram Business accounts the user administers during connection."

11.2 The screencast — where submissions actually fail

This is the single most common rejection reason. Ours was rejected on the screencast alone with **"Screencast Not Aligned with Use Case Details."** The use cases were fine; the *video* was wrong.

What made it fail: the recording showed Facebook's **returning-user dialog** ("Continue as Tabish / use previous settings") instead of the **first-time permission-grant screen listing the scopes**. The Instagram grant flashed by in a frame. No audio, no captions.

What a passing screencast must show:

- ✓ The **full permission-grant wizard** with the scope checklist visible, for **both** Facebook and Instagram.
 - If you've already connected once, Facebook shows you the returning-user shortcut. **Click "Edit settings"** to force the full wizard. This is the specific trick that fixed our submission.
 - Our provider sends `auth_type=rerequest` (`facebook.provider.ts:203`) precisely to force this wizard rather than the silent shortcut. If you're building your own client, do the same.
- ✓ The **Page-selection step**, with a Page actually ticked.
- ✓ A **real post published** to the Page and to Instagram, visible on the platform afterward.
- ✓ The **analytics/insights view** populated (click your equivalent of "Sync Now" on camera).
- ✓ **Audio narration or burned-in captions** explaining each step.
- ✓ Dead air trimmed. A 130-second publish wait reads as a broken demo — cut it.

Record generously and edit down. Loom or QuickTime → trim → burn captions. `ffmpeg` with `libass` handles caption burn-in if you have it; without `libass` you can overlay rendered PNG cues per subtitle.

11.3 Reviewer instructions

Provide:

- Test credentials** for an account inside your app (a real, working login).
- Step-by-step navigation:** where to click to connect, to publish, to see insights.
- An explicit **"click Sync Now"** step if your metrics are populated on demand — otherwise the reviewer sees an empty dashboard.
- A one-line architecture statement. Ours: *"Standard web app using Facebook Login for Business, browser-based OAuth, not server-to-server."* Make sure this does not contradict anything else in the submission.

Submit. Meta states **most submissions are reviewed within 20 days**; ours have come back in ~1–2 weeks.

12 App Review submission #2 — insights (3 permissions)

The six permissions above cover **publishing**. Reading **analytics as an external user** needs three more. Submit these separately, *after* #1 is approved and after your metric-reading code is correct.

```
pages_read_user_content
read_insights
instagram_manage_insights
```

12.1 Why these are separate — and the trap that hides the need for them

App admins, developers, and testers of your app receive every requested scope at Standard Access with no App Review. External users receive only what App Review has *approved*.

The consequence: **testing analytics on your own admin account proves nothing**. It will work perfectly and still be completely broken for every real user.

We verified this by running identical Graph calls with an admin token and a genuinely external token:

CALL	ADMIN TOKEN	EXTERNAL TOKEN
<code>GET /{post-id}?fields=reactions.summary,comments.summary</code>	200	400, error #10 — needs <code>pages_read_user_content</code>
<code>GET /{post-id}?fields=shares</code>	200	200
<code>GET /{post-id}/insights?metric=post_clicks,post_video_views</code>	200	200

CALL	ADMIN TOKEN	EXTERNAL TOKEN
GET /{ig-media-id}/insights?metric=reach	200	needs <code>instagram_manage_insights</code>
GET /{post-id}/insights?metric=post_impressions	#100	#100

Always test with a genuinely external account — one with no role on your app — before concluding a feature works for your users.

12.2 Two hard platform facts

Facebook has deleted all impression and reach metrics from the Page-post `/insights` edge. As of a live sweep on 2026-07-23, the only valid Page-post insight metrics are:

```
post_clicks
post_reactions_like_total
post_reactions_by_type_total
post_engagements
post_video_views
post_video_views_organic
```

Every `post_impressions*` variant and `post_engaged_users` return `#100 "must be a valid insights metric"`. **No permission restores them** — Meta removed them at the platform level.

Worse, Meta's `/insights?metric=` is **all-or-nothing**: including one deleted metric **400s the entire call**, zeroing out the valid metrics alongside it. Request only the six above. Our `facebook.provider.ts:366` requests exactly `post_clicks,post_video_views,post_reactions_by_type_total` for this reason.

So describe `read_insights` as covering **post_clicks and post_video_views only**. Claiming it gives you impressions or reach is factually wrong and invites a rejection.

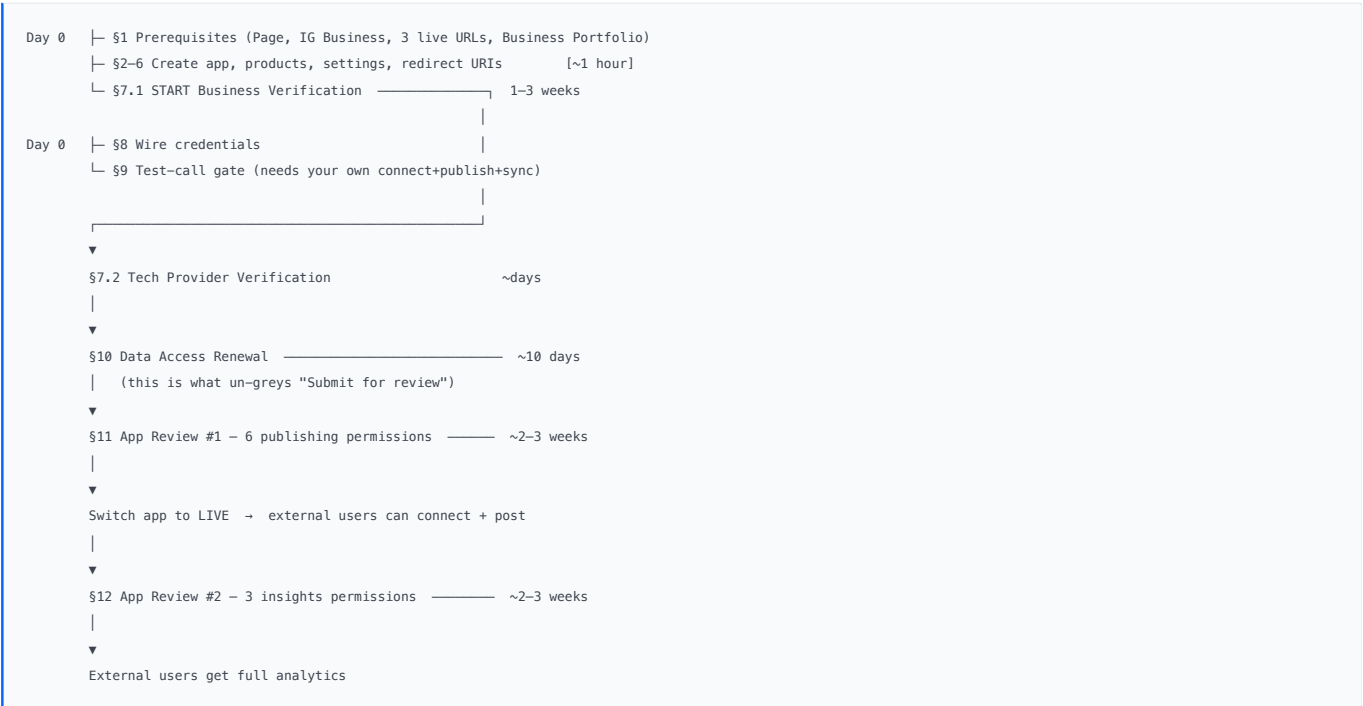
Instagram insights genuinely do need `instagram_manage_insights` for external users. Meta's Instagram Media Insights requirements table lists `instagram_basic` + `instagram_manage_insights` + `pages_read_engagement` for the Facebook-Login path. Skip it and external users' IG reach/views/saved silently store as 0 while `like_count` and `comments_count` (which ride on `instagram_basic`) look correct — a failure mode that is very easy to miss.

12.3 Honest weak point to disclose

If your app only reads insights for posts **it published**, say so. Insights on pre-existing or account-wide posts are out of scope by design — that's proper data minimization, and stating it plainly is better than having a reviewer notice your demo metrics are near zero and draw their own conclusion.

Expect near-zero numbers on fresh demo posts. Explain why in the reviewer notes.

13 Timeline: what blocks what



Total: roughly 6–10 weeks. Parallelize \$2–\$9 against the Business Verification wait; everything after that is genuinely sequential because each gate un-greys the next button.

14 Traps — do NOT do this

Each of these cost us real production time.

✗ DON'T	WHY
Leave "Native or desktop app" = ON	Server-side token exchange fails with <code>OAuthException code:1 "configured as a desktop app"</code> . See §5.
Use mixed-case or trailing-slash redirect URIs	Strict Mode does byte-exact matching. Silent OAuth failure.
Add plain "Facebook Login" instead of "Facebook Login for Business"	You lose the Page-selection wizard the whole flow depends on.
Change scopes or redirect URIs while a review is pending	Invalidates every stored user token and can reset the review. Freeze app config during any wait.
Click "Back to testing" or "Make internal" on the Audience page	Re-locks connect to your tester allowlist / your Workspace domain. There is no reason to touch these.
Request <code>instagram_manage_comments</code> for comment counts	Rejected as "Disallowed Use Case." Counts ride on <code>instagram_basic</code> .
Request any <code>post_impressions*</code> metric	Meta deleted them. Returns <code>#100</code> and 400s your entire insights call .
Test analytics only as an app admin	Admins get every scope free of review. Your test will pass and every real user will still be broken. Test externally.
Assume users have a Page	An account with no admin'd Page cannot connect. Surface an actionable error; don't show a generic OAuth failure.
Skip the test-call gate and wonder why buttons are greyed	See §9. Also allow up to 24h for propagation.
Expect a screencast of the returning-user dialog to pass	Click "Edit settings" to force the full grant wizard. See §11.2.

One more, if you're writing the client code

After **any** scope or app-config change, **all existing stored access tokens are invalidated** (`"session has been invalidated..."`). Users must reconnect via a fresh OAuth flow. Winning Advanced Access does **not** revive dead tokens — approval and token validity are independent. Plan a "reconnect required" path in your UI.

15 Reference: permission → API call → code map

Useful for writing accurate App Review descriptions and for debugging a 400.

PERMISSION	GRAPH API CALL	OUR IMPLEMENTATION
<code>pages_show_list</code>	<code>GET /me/accounts?fields=id,name,access_token,picture{url}</code>	<code>facebook.provider.ts:315 getPages</code>
<code>business_management</code>	<code>GET /me/accounts</code> (Page + IG resolution)	same
<code>pages_manage_posts</code>	<code>POST /{page-id}/feed · /photos · /videos</code>	<code>facebook.provider.ts:256 , :626 , :666</code>
<code>pages_read_engagement</code>	<code>GET /{post-id}?fields=shares,comments.summary(true),reactions.summary(true)</code>	<code>facebook.provider.ts:393</code>
<code>pages_read_user_content</code>	same fields call — 400s for external users without it	same
<code>read_insights</code>	<code>GET /{post-id}/insights?metric=post_clicks,post_video_views,post_reactions_by_type_total</code>	<code>facebook.provider.ts:366</code>
<code>instagram_basic</code>	<code>GET /{page-id}?fields=instagram_business_account</code> → IG profile read	<code>instagram.provider.ts getInstagramBusinessAccountId</code>
<code>instagram_content_publish</code>	<code>POST /{ig-user-id}/media</code> → <code>POST /{ig-user-id}/media_publish</code>	<code>instagram.provider.ts:484</code>
<code>instagram_manage_insights</code>	<code>GET /{ig-media-id}/insights?metric=...</code> (metric set varies by <code>media_product_type</code>)	<code>instagram.provider.ts:309</code>

OAuth endpoints (Graph v18.0)

```
Authorize: https://www.facebook.com/v18.0/dialog/oauth
           ?client_id=<app-id>
           &redirect_uri=<url-encoded callback>
           &scope=<comma-separated>      +- commas, not spaces
           &state=<csrf token>
           &response_type=code
           &auth_type=rerequest          +- forces the full grant wizard

Token:     https://graph.facebook.com/v18.0/oauth/access_token
           ?client_id=<app-id>
           &client_secret=<app-secret>    +- server-side only; requires $5 = 0FF
           &redirect_uri=<same callback>
           &code=<code>
```

Note Facebook joins scopes with **commas**, unlike the space-delimited convention most OAuth 2.0 providers use.

Official Meta documentation

- [Create an app](#)
- [Facebook Login for Business](#)
- [App Review](#)
- [Permissions reference](#)
- [Business Verification](#)
- [Instagram content publishing](#)
- [Page feed reference](#)
- [Graph API Explorer](#)
- [Access levels: Standard vs Advanced](#)

16 Verification checklist

Work through this before declaring the app done.

Configuration

- ☐ App type is **Business**
- ☐ Products: Facebook Login **for Business** + Instagram + Pages API
- ☐ "Native or desktop app?" = **NO**
- ☐ Privacy, Terms, and Data Deletion URLs all return **HTTP 200** in a private window
- ☐ All four redirect URLs present, lowercase, no trailing slash
- ☐ Client OAuth Login, Web OAuth Login, Enforce HTTPS, Strict Mode all **ON**
- ☐ Business Verification  · Tech Provider Verification 

Credentials

- ☐ `FACEBOOK_CLIENT_ID` == `INSTAGRAM_CLIENT_ID` == your App ID
- ☐ `FACEBOOK_CLIENT_SECRET` == `INSTAGRAM_CLIENT_SECRET` == your App secret
- ☐ `APP_URL` exactly matches the registered redirect host

Review status

- ☐ Data Access Renewal cleared
- ☐ 6 publishing permissions = **Approved / Advanced Access**
- ☐ 3 insights permissions = Approved (or knowingly pending)
- ☐ Requested scopes are an **exact match** for your code's scope arrays — zero unapproved scopes in the request

Live behavior — verify without trusting the dashboard alone:

- ☐ App Mode = **Live**
- ☐ `https://www.facebook.com/v18.0/dialog/oauth?client_id=<your-app-id>` **302s to login.php** when hit anonymously. An invalid app ID returns 200 with no login redirect — that's the control. A 302 proves Meta routes anonymous users into consent, i.e. the app is genuinely public.
- ☐ A **genuinely external** account (no role on your app) with its own Page can connect, publish to Facebook, publish to Instagram, and see non-zero insights.

That last line is the only real proof. Everything above it is necessary; only an external end-to-end post is sufficient.

Appendix: known-good production values (ours, for comparison)

Do **not** copy these — they're our credentials and domain. They're here so you can sanity-check the *shape* of your own.

SETTING	OUR PRODUCTION VALUE
App name	Post Automation 2
App ID	298449321694397
App type	Business
Graph version	v18.0
Canonical domain	https://postautomation.co.in
FB redirect	https://postautomation.co.in/api/oauth/callback/facebook
IG redirect	https://postautomation.co.in/api/oauth/callback/instagram
Approved (batch 1)	pages_manage_posts , pages_show_list , instagram_content_publish , business_management , pages_read_engagement , instagram_basic
Pending (batch 2)	pages_read_user_content , read_insights , instagram_manage_insights
Renewed	public_profile , email

Derived from the live configuration of Meta app `298449321694397` and the code that consumes it (`facebook.provider.ts` , `instagram.provider.ts` , `channel.router.ts`). Meta changes its dashboard layout and metric availability frequently — treat exact menu paths as approximate and the API-level facts as verified at 2026-07-28.